

BillBull Retail OS

# Branch & Outlet Management

Developer Implementation Guide

**Version:** 1.0.0

**Prepared for:** Engineering Team — Geebu.io

**Document Type:** Technical Implementation Specification

**Date:** May 2026

---

*Confidential — Internal Engineering Use Only*

# 01 | Feature Overview & Objectives

This document provides a complete technical specification for implementing the Branch / Outlet Management feature in BillBull Retail OS. It is intended for the backend, frontend, and database engineering teams and covers every aspect of the design, integration, and best practices required for a production-grade rollout.

## 1.1 What This Feature Does

The Branch / Outlet feature introduces multi-location awareness throughout the entire application. Every master data record, every transaction, every report, and every print action will become branch-aware. The system must correctly scope data visibility by the logged-in user's assigned branch, enforce print header logic based on transaction origin, and aggregate data under headquarters branding when an 'All Branches' scope is selected.

## 1.2 Key Objectives

- Attach a branch context to all master data — Items, Customers, Vendors, Cost Centers, Users, and Employees.
- Attach a branch context to all transactional data — Sales, Purchase, Inventory, and Financial transactions.
- Scope all reports, data grids, and filters to the active branch context of the logged-in user.
- Allow Admin and Super Admin roles to view and operate across all branches from a single login session.
- Branch-restricted users see only the data belonging to their assigned branch(es).
- Print and email outputs must reflect the header of the branch under which the original transaction was created — regardless of who reprints it.
- When generating reports with 'All Branches' selected, the report header must display Headquarters address and branding.

## 1.3 Scope of Impact

| Module           | Impacted Areas                                                       |
|------------------|----------------------------------------------------------------------|
| Master Data      | Items, Customers, Vendors, Cost Centers, Users, Employees            |
| Sales Module     | Quotations, Sales Orders, Invoices, Credit Notes, Receipts           |
| Purchase Module  | Purchase Orders, GRN, Purchase Bills, Debit Notes, Payments          |
| Inventory Module | Stock Transfer, Stock Adjustments, Physical Count, Warehouse mapping |
| Financial Module | Journal Entries, Ledger, Bank Reconciliation, Chart of Accounts      |

|                 |                                                                     |
|-----------------|---------------------------------------------------------------------|
| Reports         | All listing, summary, and detailed reports across every module      |
| Print & Email   | All print templates and email dispatch with branch-specific headers |
| User Management | Role-based branch access assignment and enforcement                 |

## 02 | Database Schema Design

### 2.1 Core Branch Table

Create a central branches table as the single source of truth for all branch data. This table must store everything needed to generate print headers, scope reports, and identify the headquarters.

| Column                  | Description                                                                              |
|-------------------------|------------------------------------------------------------------------------------------|
| id                      | Primary key — UUID or auto-increment integer. UUID is preferred for multi-tenant safety. |
| code                    | Short unique branch code (e.g. HQ, BR01, OUT-DXB). Used in transaction references.       |
| name                    | Full display name of the branch or outlet.                                               |
| address_line_1          | First line of the branch postal address.                                                 |
| address_line_2          | Second line of address (optional).                                                       |
| city                    | City of the branch.                                                                      |
| state                   | State or emirate / province.                                                             |
| country                 | Country name or ISO code.                                                                |
| postal_code             | ZIP or postal code.                                                                      |
| phone                   | Primary contact phone number for the branch.                                             |
| email                   | Branch contact email for communications.                                                 |
| fax                     | Fax number (optional, required for some print formats).                                  |
| trn_number              | Tax registration number specific to the branch, if applicable.                           |
| logo_url                | Path or CDN URL to the branch-specific logo image.                                       |
| is_headquarters         | Boolean flag. Only one record should have this set to true.                              |
| is_active               | Soft-active flag. Inactive branches are hidden from branch selectors.                    |
| sort_order              | Integer to control display ordering in dropdown lists and reports.                       |
| created_at / updated_at | Standard audit timestamps.                                                               |
| created_by / updated_by | Audit trail for who created or last modified the branch record.                          |

#### Design Note

The headquarters branch is identified by the `is_headquarters` flag, not by a hardcoded ID. This ensures the print and report logic remains dynamic and does not break if the database is seeded in a different environment.

## 2.2 Branch Column on Existing Tables

Every existing table that holds master data or transactional records must receive a `branch_id` foreign key column referencing the branches table. The following approach must be applied consistently.

| Table Category                                           | Migration Strategy                                                                                                                                  |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Master Data tables                                       | Add <code>branch_id</code> as a nullable column first, seed existing records with the headquarters branch ID, then make it NOT NULL with a default. |
| Transaction tables (Sales, Purchase, Inventory, Finance) | Add <code>branch_id</code> as NOT NULL with the headquarters branch ID as default. Apply the same seeding approach for existing data.               |
| User and Employee tables                                 | Users can belong to multiple branches. Use a junction table <code>user_branches</code> instead of a single <code>branch_id</code> column.           |
| Report configuration tables                              | Store the last selected branch filter per user as a user preference, not on the report table itself.                                                |

## 2.3 User-Branch Junction Table

Since a user may be assigned to one or more branches, avoid a single `branch_id` on the users table. Instead, introduce a junction table:

| Column                  | Description                                                         |
|-------------------------|---------------------------------------------------------------------|
| <code>id</code>         | Primary key.                                                        |
| <code>user_id</code>    | Foreign key to users table.                                         |
| <code>branch_id</code>  | Foreign key to branches table.                                      |
| <code>is_default</code> | Boolean flag indicating the user's primary/default branch at login. |
| <code>created_at</code> | Timestamp of assignment.                                            |

Admin and Super Admin role users are treated as having access to all branches and do not require records in this junction table. Their access is determined by the role flag, not by junction records.

## 2.4 Indexing Strategy

Proper indexing is critical. Every query in the application will now include a `branch_id` filter, and without indexes this will cause full-table scans on large datasets.

| Index                            | Rationale                                                   |
|----------------------------------|-------------------------------------------------------------|
| <code>branches(is_active)</code> | Speeds up all dropdown queries that filter active branches. |

|                                                             |                                                                                |
|-------------------------------------------------------------|--------------------------------------------------------------------------------|
| branches(is_headquarters)                                   | Single-row lookup for HQ header on 'All' report selection.                     |
| sales_invoices(branch_id)                                   | All sales listings filter by branch — essential.                               |
| sales_invoices(branch_id, created_at)                       | Composite index for date-range filtered branch reports.                        |
| purchase_orders(branch_id, created_at)                      | Same pattern for purchase module.                                              |
| stock_movements(branch_id, created_at)                      | Inventory queries always filtered by branch and date.                          |
| journal_entries(branch_id, created_at)                      | Financial ledger queries require both filters.                                 |
| items(branch_id)                                            | Item lookups in POS and order entry are high-frequency.                        |
| customers(branch_id)                                        | Customer search in sales is high-frequency.                                    |
| user_branches(user_id)                                      | Login-time branch lookup must be fast.                                         |
| user_branches(branch_id)                                    | Admin user listing per branch must be fast.                                    |
| [all transaction tables]<br>(branch_id, status, created_at) | Composite three-column index for filtered list views and dashboard aggregates. |

### Performance Best Practice

For very large transaction tables (100k+ rows), consider partitioning by `branch_id` if your database engine supports it. PostgreSQL table partitioning and MySQL range partitioning are both viable. Evaluate based on your data volume projections.

## 03 | Backend & API Layer

### 3.1 Authentication & Session Context

When a user logs in, the authentication layer must resolve and embed branch context into the session or JWT token. This prevents an extra database round-trip on every request.

#### 3.1.1 What to embed in the token / session

- `user_id` — unique identifier of the authenticated user.
- `role` — the user's role string (e.g. `super_admin`, `admin`, `branch_user`).
- `branch_ids` — an array of branch IDs the user has access to.
- `active_branch_id` — the currently selected branch (can be switched after login).
- `is_all_branches` — a boolean flag; true only for `admin` and `super_admin` roles.

#### 3.1.2 Branch switching

Admin and Super Admin users must be able to switch their `active_branch_id` from the UI without logging out. Expose a dedicated endpoint that accepts a `branch_id`, validates that the caller is authorized to access it, and updates the session or issues a refreshed token. Do not require a full re-login for branch switching.

### 3.2 Global Query Middleware / Service Layer

Introduce a `BranchScope` utility in the service layer that automatically appends `branch_id` conditions to database queries based on the current session context. This must be applied uniformly — do not rely on individual developers remembering to add the filter in every controller.

| Rule                                              | Implementation Approach                                                                    |
|---------------------------------------------------|--------------------------------------------------------------------------------------------|
| Super Admin / Admin role                          | No <code>branch_id</code> filter is applied. Queries return all branches' data.            |
| Admin with <code>active_branch_id</code> selected | Filter is applied with the selected branch. Admin can toggle to 'All'.                     |
| Branch User                                       | Permanently filtered to their assigned <code>branch_id(s)</code> . 'All' option is hidden. |
| 'All' branch selection (admin)                    | No <code>branch_id</code> filter. Reports use HQ header.                                   |

#### Architecture Guidance

Implement `BranchScope` as a query builder plugin or repository base class method. Every repository method that lists or aggregates records should call `BranchScope.apply(query, session)` before execution. This centralized approach ensures branch filtering is not missed across the codebase.

### 3.3 API Endpoint Design

Branch context must be communicated through every API request so the backend can apply the correct scope. Use HTTP headers rather than query parameters for consistency.

| Header / Mechanism                      | Purpose                                                        |
|-----------------------------------------|----------------------------------------------------------------|
| X-Branch-Id header on every request     | Tells the API which branch context is active for this request. |
| JWT claims for branch_ids and role      | Permanent authorization boundary — validated on every request. |
| Branch selector state in frontend store | Drives the X-Branch-Id header value sent with each API call.   |

#### 3.3.1 Endpoints to add or modify

- GET /api/branches — list all active branches (admin: all, branch\_user: only theirs).
- GET /api/branches/headquarters — returns the HQ branch record for report headers.
- POST /api/session/switch-branch — switches the active branch for the session.
- All existing list/search/report endpoints — must respect the X-Branch-Id header and apply BranchScope.
- All create/update endpoints — must write branch\_id from the session's active\_branch\_id, not from the request body (prevents spoofing).

### 3.4 Transaction Branch Locking

Once a transaction is saved, its branch\_id must be locked and immutable. No update operation should be permitted to change a transaction's branch\_id after creation, even by an admin. Enforce this in the service layer validation before any update reaches the database.

#### Critical Rule

The branch\_id on a saved transaction record is the permanent record of which branch originated that document. It is used to determine the print header at reprint time and must never be changed after the first save.



## 04 | Master Data — Branch Association

### 4.1 Items

Items (products) may be shared across branches or be specific to a branch depending on the business model. The implementation must support both patterns.

- Add `branch_id` to the items table. For items shared across all branches, allow a null or a special 'All Branches' marker value.
- The item selection dropdown in POS, sales orders, and purchase orders must filter items available to the active branch — including shared items.
- Item pricing and cost can vary by branch. Consider an `item_branch_pricing` child table if multi-branch pricing is required.
- Item stock quantities are inherently branch-specific (warehouse/location-based). Ensure stock query joins through the branch context.

### 4.2 Customers

- Add `branch_id` to the customers table — representing the branch that owns or registered the customer.
- A customer can be transacted across branches (e.g. a regular customer buys from multiple outlets). The `branch_id` on the customer is the 'owner' branch, not a restriction on which branch can sell to them.
- In the customer list view, branch users see only customers of their branch. Admins see all.
- The customer's outstanding balance and statement should aggregate across all branches if the admin is viewing, or filter to the branch if a branch user is viewing.

### 4.3 Vendors

- Apply the same ownership model as customers — `branch_id` on the vendors table identifies the branch that manages that vendor relationship.
- Purchase transactions will carry their own `branch_id` independently of the vendor's `branch_id`.

### 4.4 Cost Centers

- Cost centers must be branch-specific. A cost center belongs to one branch and should not be selectable from a different branch context.
- Add `branch_id` as a mandatory NOT NULL column on the `cost_centers` table.
- Financial transactions associating a cost center must validate that the cost center's branch matches the transaction's branch.

## 4.5 Users and Employees

- Users: use the user\_branches junction table as described in Section 2.3. Do not use a single branch\_id column on the users table.
- Employees: add branch\_id as a direct column since an employee belongs to one primary branch. If cross-branch employee tracking is needed, introduce an employee\_branches junction table on the same pattern.
- The User Management screen must allow admins to assign one or more branches to a user, set a default branch, and revoke branch access.
- An employee's payroll, attendance, and HR records should be filtered by the employee's assigned branch.

## 05 | Transactional Modules — Branch Attachment

### 5.1 General Transaction Principle

All transaction creation flows — whether initiated from POS, desktop, or API — must follow the same branch assignment rule:

1. The active branch from the user's session is assigned to the transaction at the moment of creation.
2. The `branch_id` is written to the transaction record by the backend service, not passed in from the request body.
3. Once the record is saved, the `branch_id` cannot be changed via any update operation.
4. All child records (line items, tax breakdowns, payment lines) inherit or reference the parent transaction's `branch_id`.
5. Transaction numbering sequences may be branch-specific (e.g. BR01/INV/2026/00001) — configure this in the branch settings.

### 5.2 Sales Module

- Sales Quotations, Sales Orders, Delivery Notes, Tax Invoices, Credit Notes, Receipts — all carry `branch_id`.
- The branch selector in the sales header form should be visible to admin/super admin and auto-filled (locked) for branch users.
- Customer outstanding and credit limit checks must be branch-scoped or company-wide — make this a configurable setting per the business rules.
- Sales reports (daily sales, salesperson summary, item-wise sales) must respect the active branch filter.

### 5.3 Purchase Module

- Purchase Requests, Purchase Orders, Goods Receipt Notes (GRN), Purchase Bills, Debit Notes, Vendor Payments — all carry `branch_id`.
- The receiving branch (the branch that physically receives the goods) is the `branch_id` on the GRN. This may differ from the ordering branch if inter-branch purchase flows exist.
- Vendor outstanding and aging reports must be filterable by branch.

### 5.4 Inventory Module

- Stock Entries (receive, issue, transfer, adjustment, physical count) all carry `branch_id` representing the destination/operating branch.
- Inter-branch stock transfers require TWO transaction records: a stock-out from the source branch and a stock-in to the destination branch. Both must carry their respective `branch_id` values.

- Warehouse and bin/location records must be linked to a `branch_id` so that stock level queries can be correctly scoped.
- The stock balance report must support filtering by branch. 'All Branches' should show company-wide stock with branch-wise breakdowns.

## 5.5 Financial Module

- Journal Entries, GL Postings, Bank Transactions, and Reconciliation records all carry `branch_id`.
- The Chart of Accounts may be shared company-wide or branch-specific. Decide this based on the business requirement and enforce it at the GL posting level.
- Trial Balance, Profit & Loss, and Balance Sheet reports must support branch filtering with consolidation for 'All Branches'.
- Financial consolidation (merging P&L across branches) is a separate aggregation layer — do not attempt to do this in a single SQL query without proper planning.

## 06 | Access Control & Role Enforcement

### 6.1 Role Definitions

| Role                | Branch Access               | Enforcement                                                                    |
|---------------------|-----------------------------|--------------------------------------------------------------------------------|
| Super Admin         | All branches — unrestricted | No branch filter applied in any query. Can switch active branch freely.        |
| Admin               | All branches — unrestricted | Same as super admin for data access. Can switch active branch freely.          |
| Branch Manager      | Assigned branches only      | BranchScope filters applied. Can view all data within their assigned branches. |
| Branch User / Staff | Single assigned branch only | 'All' selector hidden. Strict BranchScope enforced at API and query level.     |

### 6.2 Enforcement Points

Branch access enforcement must be applied at multiple layers, never just one. Defense in depth prevents bypass through direct API calls:

- API Middleware — validate that the X-Branch-Id in the request header is a branch the user is authorized to access. Reject with 403 if not.
- Service Layer — BranchScope.apply() is called in every repository method that reads or writes data.
- Frontend UI — branch selectors are hidden or locked based on the user's role. Branch-specific data is never rendered if not fetched.
- Report Generation — report service validates the branch parameter against session permissions before executing any query.

### 6.3 Branch Selector in the UI

A global Branch Selector component must appear in the application's top navigation bar, visible after login. Its behavior differs by role:

| User Type           | Branch Selector Behavior                                                                                                                           |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Super Admin / Admin | Dropdown showing all active branches plus an 'All Branches' option at the top. Switching updates the session context and refreshes all data views. |

|                   |                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| Branch User       | Displays the assigned branch name as static text (no dropdown). The 'All Branches' option is never shown. |
| Multi-branch user | Dropdown showing only the branches assigned to that user. No 'All Branches' option unless role permits.   |

**UX Guidance**

When an admin switches the active branch, all currently open forms or drafts should prompt a warning before the switch discards unsaved work. The branch switch should not silently clear in-progress data.

## 07 | Print Templates & Email — Branch-Aware Headers

### 7.1 The Core Rule

The print header and email sender context of any document must always reflect the branch under which the original transaction was created — regardless of who is printing or emailing it, and regardless of their current active branch context at print time.

#### Critical Logic

When a reprint is triggered, the system fetches the `branch_id` from the transaction record, then fetches the branch profile (name, address, phone, logo, TRN) from the branches table using that `branch_id`. This branch profile is injected into the print template. The admin's currently selected branch has no effect on this lookup.

### 7.2 Print Template Data Model

Each print template must accept a `branch_header` object as a parameter. The `branch_header` object contains all fields required for the header block:

- `branch_name` — full name of the branch.
- `address_line_1`, `address_line_2`, `city`, `state`, `country`, `postal_code` — complete address.
- `phone`, `fax`, `email` — contact details.
- `trn_number` — tax registration number for the branch.
- `logo_url` — branch logo image path.

At print/email generation time, the print service resolves the `branch_header` by looking up the transaction's `branch_id`, not by reading the session's active branch.

### 7.3 'All Branches' Report Header Logic

When a report is generated with the branch filter set to 'All Branches', the report header must display the Headquarters profile. The resolution logic is:

6. Check if the report's branch parameter is 'All' or null.
7. If yes, query the branches table for the record where `is_headquarters = true`.
8. Inject that headquarters branch profile as the report header.
9. If the branch parameter is a specific branch ID, fetch that specific branch profile instead.

#### Edge Case

If no branch is marked as headquarters (misconfiguration), the system should fall back to the first branch by `sort_order` and log a warning. Never allow the report to generate with an empty

or broken header.

## 7.4 Email Dispatch

- The 'From' display name in outgoing emails should reflect the branch name (e.g. 'MyCompany — Dubai Branch').
- The email body header image/logo must use the branch-specific logo\_url fetched from the transaction's branch.
- The reply-to address should be the branch email address, not a global company email, unless no branch email is configured.
- Email templates must be structured with a branch\_header injection point identical to print templates.



## 08 | Reports — Branch Filtering & Aggregation

### 8.1 Branch Filter in Report Parameters

Every report in the system must include a Branch filter parameter. The filter options available to the user depend on their role, as described in Section 6. The report service must validate this parameter against the user's permissions before executing.

| Report Type                    | Branch Filter Behavior                                                                |
|--------------------------------|---------------------------------------------------------------------------------------|
| Transaction list reports       | Filter by selected branch. 'All' returns union of all branches.                       |
| Summary/aggregation reports    | Group by branch when 'All' is selected to show branch-wise breakdown.                 |
| Financial statements (P&L, BS) | Branch-level statements and consolidated (All) statements are separate output modes.  |
| Dashboard widgets              | Each widget must respect the active branch context from the global selector.          |
| Audit log reports              | Always include branch_id in every audit record; filter by branch in the audit report. |

### 8.2 Aggregation Strategy

When reports are run with 'All Branches' selected, the underlying SQL aggregation must include a GROUP BY branch\_id to produce branch-wise subtotals before the grand total. This ensures the consolidated report is transparent and auditable.

- Never produce a single grand-total row without branch-wise breakdown for 'All Branches' reports.
- Branch names should appear as column headers or row labels in the aggregated output.
- Sort branches by sort\_order from the branches table to ensure consistent report layout.

### 8.3 Report Caching Considerations

If the application uses query result caching (Redis, in-memory, or database query cache), the cache key must include the branch\_id parameter. A cached result for Branch A must never be served to a request for Branch B.

#### Cache Key Pattern

Use a pattern such as: report:{report\_type}:{branch\_id}:{date\_from}:{date\_to}:{user\_role}. The branch\_id and user\_role are mandatory components of every report cache key.

## 09 | Frontend Implementation

### 9.1 Global Branch State

Introduce a BranchContext in the frontend application's global state management (Redux, Zustand, Pinia, or equivalent). This context holds:

- activeBranchId — the currently selected branch ID (null = All Branches for admins).
- activeBranchProfile — the full branch object for display purposes.
- availableBranches — the list of branches the user can select from.
- isAllBranches — boolean derived from activeBranchId === null and user role.

Every API call made by the frontend must read activeBranchId from this context and attach it as the X-Branch-Id request header. Use an HTTP interceptor or Axios request middleware to inject this header automatically — do not add it manually in each API call.

### 9.2 Branch Selector Component

- Place the Branch Selector component in the top navigation bar, to the right of the application logo.
- On mount, load available branches from the API and populate the dropdown.
- Selecting a branch updates BranchContext, triggers a refetch of all current page data, and persists the selection to localStorage for session persistence across page refreshes.
- Include a visual indicator (colored badge or label) to always show the current active branch at a glance.

### 9.3 Form-Level Branch Field

In all transaction creation forms (Sales Invoice, Purchase Order, Journal Entry, etc.), include a 'Branch' field:

- For branch users: show as a read-only label displaying their assigned branch.
- For admin users: show as a dropdown defaulting to the active branch context but allowing override for that specific transaction.
- The selected branch value in the form is sent to the backend but the backend must validate it against the session permissions and override it with the session's active branch if the user is not permitted to select a different one.

### 9.4 Data Table & List Views

All data grids and list views must:

- Include a 'Branch' column visible to admin users (hidden for single-branch users since all records are from their branch).
- Re-fetch data whenever the active branch in BranchContext changes.
- Display a clear label near the grid header indicating the current branch filter context (e.g. 'Showing data for: Dubai Branch').

## 10 | Migration Plan & Rollout Strategy

### 10.1 Phase 1 — Database Preparation

10. Create the branches table and insert the initial branch record(s). Mark the primary location as headquarters.
11. Create the user\_branches junction table.
12. Add branch\_id columns (nullable) to all affected master data and transaction tables.
13. Create all indexes as defined in Section 2.4.
14. Run a data migration script to backfill branch\_id = headquarters\_id for all existing records.
15. After verifying the backfill, alter the columns to NOT NULL where applicable.

### 10.2 Phase 2 — Backend Service Layer

16. Implement the BranchScope utility and integrate it into all repository base methods.
17. Modify the authentication service to embed branch context in the JWT or session.
18. Add the branch switch endpoint.
19. Update all create/update service methods to read branch\_id from the session.
20. Update all print and email generation services with the branch-header resolution logic.
21. Implement the headquarters header fallback for 'All Branches' reports.

### 10.3 Phase 3 — Frontend Integration

22. Implement the BranchContext global state and HTTP interceptor for the X-Branch-Id header.
23. Build the Branch Selector component and integrate it into the navigation.
24. Update all list views to include the Branch column and re-fetch on branch context change.
25. Update all transaction forms to include the Branch field with role-based behavior.
26. Update all report screens to include the Branch filter parameter.

### 10.4 Phase 4 — Testing

27. Unit test BranchScope for all role combinations.
28. Integration test every API endpoint with different X-Branch-Id values and roles.
29. Test the print header logic by creating a transaction under Branch A, logging in as admin under Branch B, and verifying the reprint uses Branch A's header.
30. Test the 'All Branches' report header uses HQ branding.
31. Test inter-branch stock transfer creates correct stock-out and stock-in records with correct branch IDs.
32. Test that a branch user cannot access or modify data belonging to another branch via direct API calls.

- 33. Load test branch-filtered queries on realistic data volumes to validate index performance.

## 10.5 Phase 5 — Go-Live

- 34. Deploy database migrations in a maintenance window.
- 35. Deploy backend services.
- 36. Deploy frontend build.
- 37. Assign branches to all existing users from the User Management screen.
- 38. Verify all dashboards, reports, and print formats post-deployment.
- 39. Monitor query performance via slow-query logs for the first 72 hours.

## 11 | Best Practices & Common Pitfalls

### 11.1 Things to Always Do

- Always resolve the print/email branch header from the transaction record's `branch_id`, never from the session.
- Always include `branch_id` in cache keys for any cached query result.
- Always validate the `branch_id` in API requests against the user's authorized branch list in JWT claims.
- Always apply composite indexes that include `branch_id` as the leading column alongside status and date range for transaction tables.
- Always soft-delete branches (`is_active = false`) — never hard delete. Historical transactions must still resolve their branch profile for reprinting.
- Always test branch-enforcement at the API level with direct HTTP calls, not just through the UI.

### 11.2 Things to Avoid

- Never allow `branch_id` to be passed from the frontend in a transaction creation payload and used without server-side validation.
- Never hard-code a branch ID anywhere in the application code. Always resolve dynamically.
- Never produce an 'All Branches' report that returns data without also validating the requestor has admin/super admin role.
- Never merge report data across branches in application memory — always aggregate in the database where indexes and query planners are most efficient.
- Never skip the BranchScope middleware for 'admin convenience' API shortcuts — every query must be consistent.
- Never change a transaction's `branch_id` after it is created. Enforce this in the service layer, not just the UI.

### 11.3 Audit & Compliance

- Every branch record create/update/deactivation must be written to the audit log with the acting user and timestamp.
- Every branch assignment change on a user must be audit-logged.
- Print and email events should be logged with the resolved `branch_id` of the header used, so there is a traceable record of what header was sent with each document.

## 12 | Implementation Checklist

Use this checklist to track implementation progress across teams.

### Database

- branches table created with all required columns.
- user\_branches junction table created.
- branch\_id column added to all master data tables.
- branch\_id column added to all transaction tables.
- All indexes from Section 2.4 created and verified with EXPLAIN.
- Backfill migration executed and verified.
- NOT NULL constraints applied after backfill.

### Backend

- BranchScope utility implemented and integrated into all repository base methods.
- JWT / session updated to include branch context on login.
- Branch switch endpoint implemented.
- All create operations write branch\_id from session.
- branch\_id immutability enforced in all update services.
- Print/email branch header resolution from transaction branch\_id implemented.
- Headquarters fallback logic for 'All Branches' reports implemented.
- API middleware validates X-Branch-Id against JWT claims.

### Frontend

- BranchContext global state implemented.
- HTTP interceptor injects X-Branch-Id on all requests.
- Branch Selector component built and integrated in navigation.
- Branch column added to all relevant list views.
- Branch field added to all transaction creation forms.
- Branch filter added to all report parameter screens.
- List views re-fetch on branch context change.

### Testing

- BranchScope unit tests for all roles pass.
- Reprint branch header test: transaction under Branch A reprinted by admin on Branch B shows Branch A header.
- 'All Branches' report shows HQ header.

- Branch user cannot access other branch data via direct API call.
- Index performance validated on realistic data volume.
- Inter-branch stock transfer creates correct paired records.

**Document prepared by BillBull Product Team — Geebu.io**

For questions regarding this specification, contact [engineering@geebu.io](mailto:engineering@geebu.io)